C64uRemote for M5Dial

Technical Documentation

Version 1.0

Contents

Overview **2**

Hardware **2**
- Target platform . . . 2
- Internal I²C bus . . . 3
- Free GPIOs . . . 3

Retrofitting a microSD card **3**
- Purpose . . . 3
- Choosing a module . . . 3
- Wiring . . . 4
- Supply voltage . . . 4
- Assembly . . . 4
- Bringing it up . . . 4
- Clock fallback . . . 4

Battery operation and switching off **5**
- Connecting a battery . . . 5
- Battery voltage . . . 5
- Three ways to switch off . . . 5
- Standby current and shelf life . . . 6

Build environment **6**
- PlatformIO (recommended) . . . 6
- Configuration (build_env.h) . . . 6

Wi-Fi subsystem **7**
- Runtime configuration instead of compile time . . . 7
- NVS layout . . . 7
- Card format . . . 7
- /wifi.txt on the SD card . . . 7
- Setup portal . . . 8
- Direct mode . . . 8
  - Network number (since v1.5.0) . . . 9

Software architecture **9**
- State model . . . 9
- Rendering on the round display . . . 9

ReST interface to the C64 Ultimate **10**
- Endpoints used . . . 10
- Refused connections . . . 10
- When the c64u is missing . . . 11
- Own HTTP path (since v1.4.0) . . . 11
  - NFC RF field . . . 11
  - Mesh and roaming . . . 12
  - Network diagnostics . . . 12
- Reconnecting with several networks . . . 12

CPU speed . . . 12
Joystick ports . . . 12
Disk images and autostart . . . 12
Streaming upload . . . 13

NFC subsystem **13**
Talking to the reader . . . 13
Polling strategy . . . 13
Why a dedicated probe . . . 13
Returning to the home screen . . . 14
Tag types . . . 14
Command cards . . . 14
PowerOff with confirmation . . . 15
Writing cards . . . 15
Data format . . . 15
NDEF parser: tolerance . . . 15
NFC-Info . . . 15
Copying (dump/restore) . . . 16

Interaction logic **16**
Rotary encoder . . . 16
Button . . . 16
Touch . . . 16
Blocking calls and input . . . 17
Power-off safeguards . . . 17

Persistence **17**

Project structure **17**

Troubleshooting **17**

References **18**

License **18**

How this was made **18**

Overview

C64uRemote is a firmware for the M5Dial (ESP32-S3) that remote-controls the Commodore 64 Ultimate and the Ultimate64 Elite-II through its ReST API. It also uses the RFID/NFC reader built into the M5Dial (NXP WS1850S) and a retrofitted microSD card in order to launch programs from NFC tags and to manage those tags.

The firmware is a single translation unit (`src/main.cpp`, roughly 4150 lines) built on the Arduino framework for ESP32.

Foundation: original project by Karl Prosser (github.com/ReadyOS-C64/C64uRemote), ports for the M5StickC Plus2 and M5Dial by Martin Oswald (1MHz.de). This version brings the full feature set of the M5Stack Core edition to the M5Dial.

Hardware

Target platform

Property	Value
Board	M5Dial (M5StampS3 module)
SoC	ESP32-S3, 240 MHz, 8 MB flash, **no PSRAM**
Display	240 × 240 round, GC9A01, driven through M5GFX

Property	Value
Input	Rotary encoder (GPIO 40/41), button (GPIO 42), FT3267 touch
RFID/NFC	WS1850S, built in
Also on board	BM8563 RTC, buzzer

Internal I²C bus

The M5Dial puts the touch controller, the RTC and the RFID reader on one internal I²C bus:

Device	Address	Pins
FT3267 (touch)	0x38	SDA = GPIO 11, SCL = GPIO 12
BM8563 (RTC)	0x51	SDA = GPIO 11, SCL = GPIO 12
WS1850S (RFID)	0x28	SDA = GPIO 11, SCL = GPIO 12

So unlike the Core version, the RFID reader needs **no wiring and no Unit RFID2**.

Free GPIOs

Externally the M5Dial only exposes two Grove connectors (HY2.0-4P):

Connector	Colour	Pin 1 (black)	Pin 2 (red)	Pin 3 (yellow)	Pin 4 (white)
Port A	red	GND	5 V	GPIO 13 (SDA)	GPIO 15 (SCL)
Port B	black	GND	5 V	GPIO 2	GPIO 1

Mind Port B: the yellow wire (pin 3) is GPIO 2, the white one (pin 4) is GPIO 1 - the lower number sits on the outside. Up to v1.2.1 this documentation had the two swapped.

That is exactly four free signal pins — precisely what SPI needs for an SD card, which is why both ports are used.

Retrofitting a microSD card

Purpose

The M5Dial has no card slot from the factory. Without an SD card the remote control and the tag info still work; launching programs and the NFC-Dump and NFC-Restore functions need mass storage, because the program files are read from there.

Choosing a module

Any simple SPI breakout with `VCC`, `GND`, `MISO`, `MOSI`, `SCK` and `CS` will do. Two form factors are common:

Form factor	Distinguishing feature	Supply
Breakout without regulator, approx. 18.5 × 17.5 mm	just the card holder and a pin header	**3.3 V**
Module with regulator and level shifter, approx. 42 × 24 mm	carries an `AMS1117-3.3` and a `74LVC125`	5 V

The distinction matters: the Grove connectors of the M5Dial supply **5 V**. A breakout without a regulator passes that voltage straight to the card, and micro SD cards are specified for 3.3 V only.

Wiring

SD module	M5Dial	Wire	GPIO	Constant in `main.cpp`
`SCK` / `CLK`	Port A, pin 4	white	15	`kSdSckPin`
`MOSI` / `SI` / `CMD`	Port A, pin 3	yellow	13	`kSdMosiPin`
`MISO` / `SO` / `DAT0`	Port B, pin 3	yellow	2	`kSdMisoPin`
`CS` / `SS`	Port B, pin 4	white	1	`kSdCsPin`
`GND`	Port A or B, pin 1	black	–	–
`VCC`	see below	red	–	–

The assignment can be changed in four lines at the top of `src/main.cpp`:

```
constexpr int kSdSckPin  = 15;   // Port A, pin "SCL"
constexpr int kSdMosiPin = 13;   // Port A, pin "SDA"
constexpr int kSdMisoPin = 2;    // Port B, pin 3 (yellow wire)
constexpr int kSdCsPin   = 1;    // Port B, pin 4 (white wire)
```

Supply voltage

Module with regulator: connect `VCC` directly to the 5 V of a Grove socket.

Breakout without regulator: connect `VCC` to 3.3 V. The tidiest route is a small LDO (HT7333, MCP1700, AMS1117-3.3) between the 5 V line of the Grove socket and the module's `VCC`; a 1 µF decoupling capacitor on input and output is enough. Alternatively tap 3.3 V directly on the StampS3 — that means working inside the case and is only worth it if you are soldering there anyway.

The signal lines are uncritical: the ESP32-S3 drives 3.3 V, and a breakout without a level shifter passes that through unchanged. A module with a `74LVC125` also produces valid levels from 3.3 V inputs. The only real hazard is a 5 V supply on a module without a regulator.

Assembly

1. Cut one Grove cable for Port A and one for Port B. Keep them short — the SPI lines are unshielded and run at up to 20 MHz.
2. Join both `GND` wires onto the module's `GND` pin.
3. Route the 5 V wire from Port A to the regulator input (or directly to `VCC` if the module has a regulator). The 5 V wire of Port B stays unused and should be insulated.
4. Solder the signal wires according to the table.
5. Before the first power-up, check `VCC` against `GND` for shorts and measure the voltage at the module: 3.2 to 3.4 V, or 4.8 to 5.1 V respectively.

Bringing it up

Insert a card formatted as FAT32 and start the device. The serial output (115200 baud) reports:

```
C64uRemote M5Dial v1.3.1 (2026-09-25)  RFID:1  SD:1  Canvas:1  Heap:…
```

`SD:1` means detected. On the display, every menu page shows a green **SD** at the bottom right.

Clock fallback

`initSd()` tries 20 MHz, then 4 MHz, then 1 MHz:

```
app.sdReady = SD.begin(kSdCsPin, sdSpi, 20000000);
if (!app.sdReady) app.sdReady = SD.begin(kSdCsPin, sdSpi, 4000000);
if (!app.sdReady) app.sdReady = SD.begin(kSdCsPin, sdSpi, 1000000);
```

That keeps a build with longer Grove cables working, just more slowly. The SD bus sits on **SPI3 (HSPI)**; SPI2 belongs to the display.

Battery operation and switching off

Connecting a battery

The M5Dial has a battery connector (JST 1.25 mm, 2-pin) for a single Li-ion/LiPo cell (3.7 V nominal). Charging happens via USB-C; the charger sits on the board. No on/off switch is needed, because the M5Dial switches itself:

Signal	GPIO	Function
HOLD	46	Self-hold. HIGH = stays on, LOW = disconnects the battery
WAKE / button	42	Switches on; during operation the normal button (LOW = pressed)

`M5Dial.begin()` (M5Unified) drives G46 HIGH right at the start. According to M5Stack about **1.9 µA** flow at 4.2 V while switched off.

Battery voltage

The voltage **cannot be shown without extra hardware**. The M5Dial has neither a voltage divider from the battery to an ADC pin nor a readable charger IC (its CHRG/STDBY outputs are not connected to the ESP32). M5Unified knows no PMIC for the M5Dial, so `M5.Power.getBatteryLevel()` returns **-2**. On top of that the four free GPIOs are taken by the SD card.

A fuel-gauge IC such as the **MAX17048** could be added on the internal I²C bus (G11/G12). Its address is **0x36**, so it clashes neither with touch (0x38), RTC (0x51) nor RFID (0x28).

Three ways to switch off

Long press on "c64u Power Off". `handleButton()` treats `kMenuPowerOff` in the ring menu separately: the button has to be held for `kDialOffHoldMs` (1.5 s), then `shutdownDevice()` follows directly. The generic 600 ms shortcut (back) does not apply on this icon; released earlier, `handleSelect()` runs as usual with the c64u prompt. `handleTouch()` does the same when the touch started on the icon (`menuIndexFromTouch()`); the touch shortcut is not fired there.

Command card `CMD:M5OFF` (also `CMD:DIALOFF`): `CardCmd::DialOff` calls `shutdownDevice()` without a prompt. During the first `kDialOffBootGuardMs` (8 s) after start the card is ignored (*REMOVE CARD*); otherwise a card lying on the device would switch it straight off again at every power-up. In the *NFC-Cmd* list the command appears as *M5Dial Power Off*.

Settings → "M5Dial Power Off". From v1.3.0 it sits directly below *WiFi*. Like *WiFi* it is handled in `activateSetting()` before the NFC check and therefore needs no NFC reader.

1. First press: `requestShutdown()` records the request and shows *M5DIAL OFF? AGAIN!*; the list shows *AGAIN*.
2. Second press within `kShutdownAskMs` (3 s): `shutdownDevice()`.

`shutdownDevice()` shows *OFF*, **waits until the button is released** (it would otherwise bridge the self-hold), ends SD and Wi-Fi and pulls G46 LOW. On battery the device is off at that moment.

With USB or another external supply attached the ESP32 keeps running despite HOLD = LOW. After 1.5 s the firmware therefore briefly shows *USB: sleep*, switches the display off and enters light sleep; it is woken by a GPIO wakeup on G42 (G42 is not an RTC pin, so ext0 deep-sleep wakeup is not possible there). After waking, `esp_restart()` follows. HOLD stays LOW during sleep: pulling the cable then switches the M5Dial off completely.

Alternatively the **RST button** on the back also switches off on battery, because G46 is no longer driven during the reset.

Standby current and shelf life

Switched off, the M5Dial draws about 2 µA according to the datasheet; the protection circuit of a typical cell adds 1 to 6 µA. For a 650 mAh battery that is below 0.1 mAh per month in total - negligible. What counts is the cell's own **self-discharge** (roughly 1 to 3 % per month). Fully charged, the battery therefore lasts many months switched off; after half a year it should still be above 80 %.

Precondition: the SD module hangs on the 5 V line of the Grove connectors and is switched off with it. Measuring once is worthwhile: multimeter in µA/mA mode in the battery's positive lead, switch the M5Dial off - it should show a few µA.

Build environment

PlatformIO (recommended)

The configuration is version-controlled in `platformio.ini`, which makes builds reproducible.

Setting	Value	Reason
`board`	`m5stack-stamps3`	The M5Dial is a StampS3
`platform`	`espressif32@^6.9.0`	Arduino framework for the ESP32-S3
`board_build.partitions`	`huge_app.csv`	Wi-Fi + SD + RFID exceed the default app partition
`build_flags`	`-DARDUINO_USB_CDC_ON_BOOT=1, -DARDUINO_USB_MODE=1`	native USB CDC for the serial monitor
`monitor_speed`	115200	serial output

Libraries (`lib_deps`):

- `m5stack/M5Dial` (^1.0.2) — display, touch, encoder **and** the RFID driver (`MFRC522`)
- `m5stack/M5Unified` (^0.2.8) and `m5stack/M5GFX` (^0.2.11)
- `bblanchon/ArduinoJson` (^6.21.5) — deliberately pinned to version 6, since version 7 no longer has `DynamicJsonDocument`

A separate MFRC522 library as used by the Core version is **not** needed: the M5Dial library ships a class `MFRC522` whose API matches the `MFRC522_I2C` used there.

Commands:

```
pio run                 # compile
pio run -t upload       # flash
pio device monitor      # serial output
```

If no serial port shows up, put the StampS3 into download mode by holding the button while plugging it in, flash, then reset once.

Configuration (build_env.h)

`build_env.h` only supplies the **initial values** now. As soon as a network configuration is present in NVS, the file is ignored. Copy the template `build_env.h.example` to `build_env.h` and fill it in:

```
#define C64U_WIFI_SSID       "MyWLAN"
#define C64U_WIFI_PASSWORD   "MyPassword"
#define C64U_TARGET_HOST     "192.168.0.64"
#define C64U_TARGET_PASSWORD ""       // only if set on the C64
```

Without the file the project still compiles with empty defaults; the M5Dial then shows *SETTINGS > WIFI* at startup and can be set up on the device.

Wi-Fi subsystem

Runtime configuration instead of compile time

SSID, password, c64u address and c64u password live in NVS at runtime and can be changed through *Settings → WiFi*. Up to `kWifiProfileMax` (= 4) profiles are kept:

```
struct WifiProfile { String ssid; String pass; };
WifiProfile gWifiProfiles[kWifiProfileMax];
size_t      gWifiCount;
size_t      gWifiTry;      // profile for the next connection attempt
```

`beginWiFi()` takes `gWifiProfiles[gWifiTry]` and then advances the index by one. If a connection fails, `serviceWiFi()` tries the next profile after `kWiFiRetryMs` – across several rounds the attempt therefore walks through all known networks. At start-up `wifiPickBestProfile()` scans the area once and begins with the strongest known network.

`wifiAddProfile()` sorts a new network in at the front; a network that is already known merely gets a new password and moves to the front as well. The oldest profile drops off the back if need be.

NVS layout

Namespace `c64unet`, separate from the interaction settings in `c64udial`:

Key	Contents
`wn`	number of profiles (0…4)
`s0…s3`	SSID
`p0…p3`	password
`host`	address of the c64u
`hpass`	password of the c64u
`dmode`	direct mode on/off
`dssid`, `dpass`	SSID and password of the direct network
`dnet`	address range of the direct network, e.g. `192.168.4`

Factory Reset only touches `c64udial`; the network configuration survives and is discarded exclusively through *WiFi → Delete all*.

Card format

Wi-Fi cards use the same scheme as Wi-Fi QR codes, stored as an ordinary NDEF text record:

`WIFI:S:<ssid>;T:WPA;P:<password>;;`

`parseWifiText()` evaluates the `S` and `P` fields, accepts backslash-escaped special characters and additionally understands the short form `WIFI:<ssid>;<password>`. `wifiCardText()` is the counterpart and is used by *WiFi → To NFC card*.

On the setup page (`ScreenMode::WifiCard`) a card text without the `WIFI:` prefix counts as a plain password for the network picked beforehand.

Outside the setup – that is, on the home screen, in the ring menu and on the read page – `processCard()` recognises a complete `WIFI:` card, stores the network and connects right away. `gWifiTry` is pointed at that profile on purpose and the code waits up to `kWifiCardConnectMs` (8 s) for `WL_CONNECTED`; after that `serviceWiFi()` takes over again. If the network is already connected the function bails out with *ALREADY CONNECTED* – that prevents an endless loop when the card stays on the reader and the background poll spots it again.

/wifi.txt on the SD card

`loadWifiFromSd()` reads a simple key/value format. Every new `ssid` line begins an entry; `#` and `;` introduce comments. `host` and `hostpass` are recognised as well.

```
ssid = MyWiFi
pass = secret

host     = 192.168.0.64
hostpass =
```

The file is read at start-up (only while `gWifiCount == 0`) and on demand through *WiFi → Load from SD*.

`saveWifiToSd()` is the counterpart: it writes all profiles together with `host` and `hostpass` back to `/wifi.txt` with a comment header. An existing file is moved to `/wifi.bak` with `SD.rename()` beforehand; if that fails it is deleted. The function returns the number of networks written and puts a plain-text hint into `errorOut` on failure.

Setup portal

`startPortal()` switches to `WIFI_AP`, opens an access point on fixed channel 1 (`kPortalSsid` / `kPortalPass`), starts a `DNSServer` as a captive-portal redirect and a `WebServer` with two routes (`/` and `/save`). The station part is deliberately shut down: if it stayed active it would keep looking for the stored network in the background, change the radio channel while doing so and drop clients that had joined.

`servicePortal()` runs on every loop pass, holds the idle clock while a client is connected, and ends the portal after `kPortalIdleMs` (5 min) or `kPortalCloseMs` after a successful save. `stopPortal()` restores `WIFI_STA` and reconnects immediately.

Direct mode

For meetings without a router the M5Dial opens a WiFi network of its own (`WIFI_AP`, without a station part – for the same reason as the setup portal: a searching STA changes channel and throws connected devices off).

SSID / password	`C64uRemote-Direct` / `c64ultimate` (defaults, changeable)
Channel, max. clients	6, 8
M5Dial address	`<net>.1`
DHCP range	`<net>.64` … `<net>.74`
c64u target	`<net>.64`
`<net>`	`192.168.4` (default) or `192.168.2`, anything via `direct_net`

Start order: `WiFi.softAP()` first, then `WiFi.softAPConfig(ip, ip, 255.255.255.0, <net>.64)`. The fourth parameter sets the start of the DHCP range (Arduino core 2.0.17 hands out eleven addresses from there). The other way round, depending on the core version, the default `192.168.4.1` may stay – the setup portal never notices because it uses exactly that address.

Target address: in direct mode `targetHost()` returns `gDirectHost` instead of the stored home address, which stays untouched. If the device is connected as a normal client to a network whose SSID matches the direct network's, `<net>.64` applies as well – so a second remote can join without changing its settings.

Finding the c64u: the first DHCP lease is `.64`. If another device was quicker, `directNextCandidate()` keeps looking: candidates are `.64` and all addresses from `esp_wifi_ap_get_sta_list()` + `esp_netif_get_sta_list()`. After each failed status test the next one is tried. A device counts as the c64u if it answers `/v1/version` with JSON (or with 401/403 if a password is missing). Once an address has answered, it is only switched after the second failure in a row – the c64u occasionally refuses on its own (see *Refused connections*).

Status test: with nobody connected the test is skipped entirely. When a device joins (`WiFi.softAPgetStationNum()` rises) and the c64u is not confirmed yet, it is checked right away, starting again at `.64`. Since the c64u is often still busy with DHCP at that point, up to six follow-up checks follow at 3 s intervals (`directRecheckDue()`), on any page, until it answers. In the direct network the connection setup also has a 1 s deadline.

Resetting DHCP: the ESP32's DHCP server hands out addresses via a pointer that only moves forward. If the c64u releases its address when leaving or asks for the old one when coming back, the server drops the entry and hands out the next one (`.65`, `.66` …). So `directRestartDhcp()` restarts the server as soon as the last device has left – afterwards it starts at `.64` again. With other devices still connected this is skipped, because the server forgets all assignments and could otherwise hand out an address twice; the search then finds the c64u at its new address.

Switching: `setDirectMode()` stores the setting, discards the connection state and restarts AP or WiFi. An explicitly chosen network (list, scan, WiFi card) ends direct mode. The setup portal displaces the direct AP temporarily; after the portal, `beginWiFi()` starts it again.

Scanning in direct mode: only the station part can scan, so for the duration of the scan the mode switches briefly to `WIFI_AP_STA` and then back to `WIFI_AP`.

NVS (namespace `c64unet`): `dmode` (bool), `dssid`, `dpass`, `dnet`. Unusable values (SSID empty or > 32 characters, password < 8 or > 63, address range not three numbers 0…255) are replaced by the defaults on load.

Network number (since v1.5.0)

Several devices in direct mode at one table need different network names. The number lives solely in the network name `gDirectSsid` (NVS `dssid`): `directNumFromSsid()` returns 0 for `C64uRemote-Direct`, 1–9 for `C64uRemote-Direct-1` … `-9` and −1 for a freely set name. `directChannel()` picks the radio channel for `WiFi.softAP()` from it: without a number `kDirectChannel` (6) as before, otherwise 1, 6, 11 in turn. The menu entry *Direct no.* (`kWifiDirectNum`) steps 0 → 1 … 9 → 0 via `setDirectNum()`, saves and restarts a running access point.

Software architecture

State model

All runtime state lives in the global `struct AppState app`. The display follows a screen enum:

```
enum class ScreenMode {
  Home, Menu, CpuMenu, Status, Settings, SdBrowser,
  RfidRun, RfidWrite, RfidInfo, RfidDump, RfidRestore, Busy
};
```

Compared to the Core version, `Menu` is new: the M5Dial has no command tiles on the home screen but a ring menu of its own.

The main loop runs at roughly 30 fps (`kFrameMs = 33`):

1. `M5Dial.update()` — refresh button, touch and encoder
2. `serviceWiFi()` / `refreshConnectionStatus()` — maintain the network
3. `handleEncoder()`, `handleButton()`, `handleTouch()` — evaluate input
4. `serviceRfid()` — RFID polling (every 250 ms, only on RFID screens)
5. `updateHomeDemo()` — animation sequence and the return to the home screen
6. `render()` — compose and push the frame

Rendering on the round display

Connection status in the ring menu. `drawRoundFrame(withStatus, withDot)` draws the status dot at y = 14. In the ring menu it would sit right under the *c64u Power Off* icon, so `drawMainMenu()` calls `drawRoundFrame(true, false)` and colours the Status icon (`kMenuStatus`) with `connectionColor()` instead - the glyph always, the frame only while it is not selected.

The StampS3 has no PSRAM. A full-screen buffer at 240 × 240 × 2 bytes takes 115 kB and does fit in the heap, so the firmware allocates it as an `M5Canvas` and pushes one finished frame per cycle. That avoids flicker and makes the dirty flag logic of the Core version unnecessary.

If `createSprite()` fails, `gDraw` points at `M5Dial.Display` and drawing happens directly:

```
gUseCanvas = (canvas.createSprite(kScrW, kScrH) != nullptr);
gDraw = gUseCanvas ? static_cast<lgfx::LovyanGFX*>(&canvas)
                   : static_cast<lgfx::LovyanGFX*>(&M5Dial.Display);
```

All drawing routines address `gDraw` only, so the fallback is invisible to the rest of the code.

Logo at 1:1. The logo in `1MHz_logo_rgb565.h` is 240 × 135 pixels and therefore matches the display width exactly. Unlike the Core version there is no scaling and no scaling table; `logoPixel()` reads straight from flash via `pgm_read_word`.

Circular mask. Instead of testing the distance to the centre for every pixel, the half chord width is computed once per row:

```
int chordHalfWidth(int y) {
  const int dy = y - kCy;
  const int inner = kRadius * kRadius - dy * dy;
  return inner <= 0 ? 0 : (int)sqrtf((float)inner);
}
```

The same function bounds the text width of every line so labels are not clipped by the round edge.

Effects. `drawDistortedRows` (water/sine), `drawRotoZoom`, `drawRipple` and `drawRasterBars` come from the Core version and work row by row on the 240 × 135 logo area.

ReST interface to the C64 Ultimate

All commands go through the HTTP ReST API of the Ultimate firmware (3.11 and later). Base URL: `http://<host>/v1/…`. If a network password is set it is sent in the `X-Password` header.

Endpoints used

Purpose	Method	Path
Reachability / auth	GET	`/v1/version`
List categories	GET	`/v1/configs`
Read a setting	GET	`/v1/configs/<category>/<item>`
Write a setting	PUT	`/v1/configs/<category>/<item>?value=…`
Reset	PUT	`/v1/machine:reset`
Reboot	PUT	`/v1/machine:reboot`
Ultimate menu	PUT	`/v1/machine:menu_button`
Power off	PUT	`/v1/machine:poweroff`
Write memory	PUT	`/v1/machine:writemem?address=…&data=…`
Read memory	GET	`/v1/machine:readmem?address=…&length=…`
Query drives	GET	`/v1/drives`
Mount an image	POST	`/v1/drives/<a|b>:mount?type=…&mode=readwrite`
Switch a drive on	PUT	`/v1/drives/<a|b>:on`
Run a program	POST	`/v1/runners:run_prg`
Run a cartridge	POST	`/v1/runners:run_crt`
Play a SID	POST	`/v1/runners:sidplay`
Play a MOD	POST	`/v1/runners:modplay`

Refused connections

The HTTP server of the Ultimate firmware accepts only one connection at a time and refuses further ones with a TCP RST; `HTTPClient` reports this as *connection refused*. It was observed with only a single device on the network as well - so it happens sporadically and is neither a radio nor an address problem.

The actual call lives in `sendApiRequestOnce()`. `sendApiRequest()` is a wrapper around it: if no connection comes about (`HTTPC_ERROR_CONNECTION_REFUSED`), up to two more attempts follow after `kApiRetryDelayMs` (`kApiConnectAttempts` = 3). Retrying happens **only** in this case - nothing

reached the c64u then, so a command cannot be doubled. Read errors and HTTP error statuses (4xx, 5xx) are passed through unchanged.

On top of that `refreshConnectionStatus()` saves a request: without a stored password the second query would be byte-identical to the first, because the `X-Password` header is only set when there is one. That halves the base load on the c64u.

When the c64u is missing

If the c64u is off or cannot be found at the address, no TCP RST comes back but nothing at all. `HTTPClient` then waits until its connect limit runs out, which is 5 s by default. Up to v1.3.0 the main loop therefore stood still for about ten seconds per status query (two attempts), and with it display, dial and touch.

Since v1.3.1:

- `sendApiRequestOnce()`, `readC64Byte()` and `uploadFile()` connect with a short limit (now `kHttpConnectTimeoutMs` = 1.5 s per attempt, see *Own HTTP path*).
- `sendApiRequest()` only retries if no connection came about at all.
- While the c64u counts as unreachable, the periodic status query opens a non-blocking socket to port 80 with `probeStart()` and checks it on every loop pass with `probePoll()` via `select()` without waiting. Once connected, the pre-check sends a complete request and reads the reply to the end. If the c64u answers (connection accepted or `ECONNREFUSED`), the actual query follows in `probeTargetNow()`; after `kProbeConnectMs` (4.5 s) without an answer it stays at *Target unreachable*. Leaving the home screen or status page aborts the pre-check. With a host name instead of an IP address the pre-check is skipped, because the name lookup itself blocks.
- `loop()` only loads the CPU speed once `app.connection.targetReachable` is set.

Own HTTP path (since v1.4.0)

All requests to the c64u - `sendApiRequestOnce()`, `readC64Byte()` and the uploads in `uploadFile()` - no longer go through `HTTPClient`/`WiFiClient` but through small functions on lwIP sockets:

- `rawOpen()` connects without blocking and checks via `select()` without waiting every 2 ms; limit `kHttpConnectTimeoutMs` (1.5 s), in direct mode `kHttpConnectDirectMs` (1 s).
- `rawSendAll()` sends the request or file in pieces with `MSG_DONTWAIT`.
- `rawReadResponse()` reads the reply until the c64u closes the connection (or shortly after the last byte announced by `Content-Length`) and evaluates the status line, `Content-Length` and `chunked`. If the c64u closes without a reply the error is *connection lost*, if the reply does not come, *read Timeout*.

`rawHttpRequest()` combines this for GET/PUT; every request carries `Connection: close`. Background: with `HTTPClient` most requests got lost on the home network at times, while a connection checked without waiting got through reliably.

`staSsid()` and `staRssi()` return network name and signal strength from a cache that is refreshed at most once per second via `esp_wifi_sta_get_ap_info()`. `WiFi.SSID()` asks the driver on every call, and `targetHost()` needs the network name all the time (detecting the direct network as a client). `directStationCount()` asks for the number of stations in direct mode at most every 500 ms.

NFC RF field

The MFRC522 driver leaves the 13.56 MHz field on permanently after `PCD_Init()`. `rfidFieldOn()` and `rfidFieldOff()` switch it on only for the card probe (`cardPresent()`, `cardPresentQuick()`, then 5 ms for the card to power up) and while a detected card is being processed; on an empty probe it goes off again. It is also switched off before every network request (`rawOpen()`, `probeStart()`) - according to M5Stack the M5Dial's RFID and WiFi share the antenna, and WiFi is blocked while the field is on. So that a card left on the reader is not executed or written again every time the field comes on, `rfidHoldCard()` remembers its UID after `processCard()`. `rfidHeldCardGone()` switches the field on briefly on every check, reads the UID and switches off again right away; the same card is ignored, only two misses in a row or a different card clear

the way. On the M5Dial the NFC antenna sits right next to the WiFi antenna: with the field permanently on and the background check running, packets to the M5Dial kept getting lost for seconds, queries and uploads failed.

Mesh and roaming

`beginWiFi()` sets `WIFI_ALL_CHANNEL_SCAN` and `WIFI_CONNECT_AP_BY_SIGNAL`: the M5Dial scans all channels and connects to the strongest access point of the network instead of the first one found. `serviceRoaming()` watches the signal strength during operation; if it stays below `kRoamRssiDbm` (-72 dBm) for longer than `kRoamWeakMs` (20 s), a background scan starts. If another access point of the same network is at least `kRoamGainDb` (8 dB) stronger, the M5Dial switches to it via `WiFi.begin(..., channel, BSSID)`. At least `kRoamGapMs` (3 min) lie between two scans.

Network diagnostics

With `kNetDiag = true` the firmware prints lines with `[NET ...]` on the serial port: every HTTP request with result and duration, the pre-check, WiFi events and every 10 s the network state (access point, channel, RSSI, IP, the c64u's ARP entry, free memory). It is off by default.

Reconnecting with several networks

`beginWiFi()` moves on to the next stored profile after every attempt. Without a countermeasure that means: with two networks stored of which only one is reachable, every other reconnect after a dropout hits the dead one and costs a full retry cycle (`kWiFiRetryMs`, 10 s).

`serviceWiFi()` therefore remembers the profile the connection came up on, via `wifiProfileIndex(staSsid())`, and makes it the next attempt; `gWifiNoted` keeps this to once per connection. After a dropout the first attempt goes back to the working network, and the dead one is only tried if the good one is really gone.

CPU speed

The setting sits under a different path depending on the firmware. Therefore `resolveCpuPath()` first looks specifically in the category *U64 Specific Settings* and otherwise walks all categories until it finds an item whose name contains both "CPU" and "Speed". The list of choices comes from the `values` field of the response; if that fails, a built-in list is used.

Joystick ports

There is no `machine:` command for swapping the joystick ports. The mapping is a configuration item - in testing *Joystick Swapper* in the category *U64 Specific Settings*, with the values `Normal`, `Swapped`, `WASD Port 2` and `WASD Port 1`. It is therefore set through `/v1/configs/<category>/<item>?value=…`, exactly like the clock speed.

As with the clock, `resolveJoyPath()` looks in *U64 Specific Settings* first and otherwise walks all categories until an item name contains "Joystick", so a later firmware renaming it goes unnoticed. `refreshJoyChoices()` reads `values` and `current`.

`joyTokenFromValue()` and `joyValueFromToken()` convert between the device value and the card token (`WASD Port 1` <-> `WASD1`), so a card needs neither a blank nor a firmware-specific spelling. `toggleJoystickSwap()` toggles between `Normal` and `Swapped` and returns to `Normal` from a WASD mode; `cycleJoystickValue()` walks through every reported value in the settings menu.

Disk images and autostart

When mounting a disk image, `type` and `mode` belong in the query and the file is the only multipart part. The Ultimate firmware treats every multipart part as an attachment; a leading text field results in *Invalid type*.

With `Disk Action = Mnt+Run` the following runs afterwards:

1. `PUT /v1/drives/<drive>:on` — switch the drive on or reset it

2. `PUT /v1/machine:reset`
3. wait for the BASIC prompt
4. write `LOAD"*",8,1` into the keyboard buffer
5. wait for loading to finish
6. write `RUN` into the keyboard buffer

Rather than using fixed delays, address `$CC` (BLNSW) is polled through `readmem`: a value of 0 means the cursor is blinking, so BASIC is waiting for input. Typing goes through `$0277…$0280` (keyboard buffer) and `$C6` (character count), with `$C5` cleared beforehand. Since only ten characters fit in the buffer, the PETSCII abbreviations `lO` and `rU` are used.

Streaming upload

`uploadFile()` opens its own `WiFiClient` connection, writes the HTTP headers by hand and pushes the file to the socket in 1 kB blocks straight from the SD card. A 175 kB `.d64` therefore needs no buffer of its own size — which matters on a device without PSRAM whose heap also carries the 115 kB frame buffer.

`publishProgress()` refreshes the progress bar every 150 ms.

NFC subsystem

Talking to the reader

The reader is initialised together with the encoder:

```
M5Dial.begin(cfg, true, true);   // encoder, RFID
```

It is addressed through `M5Dial.Rfid` (class `MFRC522`). Detection reads the version register; `0x00` and `0xFF` mean “not responding”:

```
const uint8_t version = RFID.PCD_ReadRegister(MFRC522::VersionReg);
```

Polling strategy

`serviceRfid()` runs in two modes:

1. **On the RFID screens** (`RfidRun`, `RfidWrite`, `RfidInfo`, `RfidDump`, `RfidRestore`) a full `cardPresent()` poll happens every 250 ms.
2. **On the home screen and in the ring menu** a quick probe with `cardPresentQuick()` runs at the interval configured under *Auto-NFC*. When it finds a tag, the code sets `autoRfidActive`, switches to `RfidRun` through `setScreen()`, draws one `render()` and then calls the same handling the reading screen uses.

The actual tag handling lives in `processCard()`. Both paths call it, so there is only one implementation.

Why a dedicated probe

With no tag present, the MFRC522 waits after the REQA command until its internal timer expires — about 25 ms by default. The main loop stalls for exactly that long, which would show up as a stutter while an animation is running.

A tag answers far more quickly: the frame delay time at 106 kbit/s is roughly 86 µs. `cardPresentQuick()` therefore shortens the window to about 2 ms for the probe only and restores it immediately afterwards — before `PICC_ReadCardSerial()`. Selection, authentication and every write operation still run with the full window.

```
void setRfidTimerReload(uint16_t ticks) {          // 1 tick = 25 µs
  RFID.PCD_WriteRegister(MFRC522::TReloadRegH, ticks >> 8);
  RFID.PCD_WriteRegister(MFRC522::TReloadRegL, ticks & 0xFF);
}
```

The original value is not hard-coded but read from `TReloadRegH/L` in `initRfid()` and kept in `gRfidTimerReload`. Should a future library version change the default, the behaviour stays correct.

Cost. A probe that finds nothing consists of a handful of I²C register accesses plus the roughly 2 ms wait, some 5 ms in total. At the default interval of 700 ms that is a background load below one per cent, not enough to miss a 33 ms frame.

Auto-NFC	Interval	approximate load
Off	–	0 %
1.5s	1500 ms	~0.3 %
0.7s	700 ms	~0.7 %
0.3s	300 ms	~1.7 %

Returning to the home screen

So the screen saver takes over again after an automatic launch, `app.autoRfidActive` records that the reading screen was not opened by the user. Only then does the *Home Timeout* apply to it. As soon as the user navigates, `setScreen()` clears the flag.

If *Home Timeout* is set to *Off*, `kAutoRfidHoldMs` (20 s) applies instead for this case, so an automatically opened screen never stays up indefinitely.

Tag types

Type	Detection	Layout
MIFARE Classic 1K/4K/Mini	SAK via `PICC_GetType()`	16-byte blocks from block 4, sector trailers skipped
NTAG213/215/216, Ultralight	SAK 0x00	4-byte pages from page 4

For MIFARE Classic, authentication is attempted first with the NDEF key `D3F7D3F7D3F7`, then with the factory key `FFFFFFFFFFFF`. The key that worked last is remembered so that not every block costs a failed attempt. After a failed authentication the card is in HALT state and has to be selected again through `PICC_WakeupA()` — that is what `reselectCard()` does.

Command cards

If a card carries the prefix `CMD:` instead of a file path, its content is executed as a command for the c64u. Neither an SD card nor a file is needed.

```
CMD:RESET
CMD:REBOOT
CMD:MENU
CMD:POWEROFF=0      power off immediately
CMD:POWEROFF=8      ask first, 8 s confirmation window
CMD:POWEROFF        ask first, using the device setting "NFC-Cmd PowOff"
CMD:M5OFF           switch the M5Dial itself off (also CMD:DIALOFF)
CMD:CPU=10          set the CPU to 10 MHz
CMD:JOY             toggle the joystick ports (Normal <-> Swapped)
CMD:JOY=SWAPPED     set the ports fixed; also NORMAL, WASD1, WASD2
```

`parseCardCommand()` takes the text apart: check the prefix, split off an optional argument after `=`, compare the keyword in upper case. Whitespace and letter case do not matter. The content stays a plain NDEF text record, so any NFC app can read and write such a card.

In its reading branch `processCard()` checks for a command first and calls `runCardCommand()`. If a card carries the prefix but no known keyword, the device reports *BEFEHL UNBEKANNT* instead of turning it into a file path.

PowerOff with confirmation

The waiting time is an argument **on the card**, not in the device; `cardPowerOffSeconds()` returns it. Without an argument the setting *NFC-Cmd PowOff* applies (3/5/8/15 s), `0` means “no prompt”.

The sequence uses three fields in `AppState`:

```
bool     cardPowerOffPending;
String   cardPowerOffUid;      // only the same card confirms
uint32_t cardPowerOffUntilMs;
```

Confirmation happens by presenting the same card again or by pressing the button. A different card cancels, and so does letting the window expire - both clear the flag without triggering anything. Binding to the UID prevents an unrelated card placed nearby from powering the machine down.

Writing cards

`cmdListAt()` builds the selection list: five fixed commands, then every CPU step currently held in `app.cpuDisplayOptions` (loaded from the c64u, otherwise the built-in fallback list). `cardCommandText()` turns the choice into the card text. The screen `ScreenMode::CmdPick` shows the list; the selection ends up in `app.pendingCardText`, which the writing screen prefers over `pathToCardText(app.pendingPath)`.

Data format

Compatible with TeensyROM and Zaparoo/TapTo: a single NDEF record of type *Text* (Well Known, UTF-8, language code “en”).

```
TLV     : 03 <len> … FE
Record  : D1 01 <plen> 54 | 02 'e' 'n' | <text>
```

The text is the path to the program file, for example:

```
SD:OneLoad v5/Bubble Bobble.crt
```

Allowed prefixes are `SD:`, `USB:`, `TR:` or none at all. `cardTextToPath()` strips the prefix, collapses duplicate slashes and guarantees a leading slash. A `?` as the file name, or a plain directory path, triggers a random pick through `resolveRandomFile()`.

In addition, `readCardContent()` still recognises the older raw format marked `C64UPATH`, so tags written earlier keep working.

NDEF parser: tolerance

Some writers store an incorrect payload length — the TeensyROM, for instance, always writes `0x10`. For the last record (ME flag set) the size derived from the TLV length therefore takes precedence. On top of that `sanitizeCardText()` drops every character below 0x20, because some apps append NUL, CR or LF, which would otherwise ruin any file extension.

NFC-Info

`collectCardInfo()` gathers the UID, SAK, type, memory size, the result of `GET_VERSION` (0x60) and the tag contents. `collectCardRaw()` adds raw data: for NTAG the pages 0–15, lock bytes, capability container, the password protection from the configuration page and the read counter (`READ_CNT`, 0x39); for Classic the blocks 0, 4, 5, 6 and 8 plus the key in use.

`buildInfoView()` wraps the result to 30 characters per line and stores it in a scrollable list — eleven lines fit on a 240 × 240 display at once.

Commands that can deselect a tag are deliberately placed last: if `GET_VERSION` fails, a `reselectCard()` follows immediately.

Copying (dump/restore)

`dumpCardToSd()` writes a text file to `/NFC-DUMPS/<UID>.nfc`:

```
# C64uRemote NFC-Dump
type NTAG215
uid  04 01 A1 01 C1 47 03
sak  00
P4   03 27 D1 01
```

For MIFARE Classic, a key dictionary of 13 common keys is tried per sector (`classicAuthDict()`), so foreign tags can be backed up as well. Since key A can never be read back, the dump writes the key that actually worked into the trailer.

`restoreDumpToCard()` writes back user data only:

- NTAG/Ultralight from page 4 up to the last user page; pages 0–3 (UID, lock, capability container) and the configuration pages stay off limits
- Classic without block 0; sector trailers only when `validAccessBits()` confirms the access bit pattern is self-consistent

Both restrictions prevent a tag from being bricked. The UID itself is fixed in the chip and is not copied.

Interaction logic

Rotary encoder

`M5Dial.Encoder.read()` produces very fine tick sequences. `handleEncoder()` accumulates them in `app.encoderResidual` and emits a menu step only after the configured number of ticks (*Encoder Steps*: 1, 2, 4 or 6). On the home screen the first turn only opens the menu without moving the selection.

Button

The M5Dial has only one button, so the button combinations of the Core version are gone:

Event	Effect
short press	`handleSelect()`
long press (≥ 600 ms) on the home screen	`runShortcut(shortcutButton)`
long press elsewhere	`handleBack()`

The flag `app.buttonHandled` makes sure that after a long press has been handled, the release does not trigger a second action.

Touch

`handleTouch()` records position and time on the first detected contact and evaluates on release. On the home screen and in the ring menu, holding for 600 ms triggers `runShortcut(shortcutTouch)`; the release tap is then skipped.

Hit testing in the ring menu works in two stages (`menuIndexFromTouch()`):

1. a direct hit within `kMenuHitRadius` = 30 px of an icon
2. otherwise the angle to the centre via `atan2()`, split into ten 36° sectors

The whole ring outside `kMenuRingInner` = 50 px is therefore active. Inside `kMenuCenterRadius` = 50 px sits the logo, and a tap there returns to the home screen. In lists, the gap between two rows belongs to the row above so there are no dead strips.

Blocking calls and input

The periodic connection test costs two HTTP calls with a 3 s timeout each and blocks the main loop accordingly; while it runs, the touchscreen is not polled. `refreshConnectionStatus()` therefore performs the periodic test only on the home screen and the status page. During interaction it runs on explicit request only (`force = true`), that is when opening the status page and on a press there.

Power-off safeguards

Two separate mechanisms:

- **c64u Power Off menu entry:** `requestPowerOff()` sets `pendingPowerOff`; only a second press within *PowerOff Zeit* powers down. This always applies.
- **PowerOff shortcut:** depending on *PowerOff Abfrage* either immediately or with a confirmation prompt (`comboPowerOff`) acknowledged by another press.

The M5Dial itself has its own safeguard, see *Battery operation and switching off*: press twice in the settings, hold for 1.5 s in the ring menu.

Persistence

Settings live in NVS under the namespace `c64udial` (the Core and StickC editions use `c64uremote`, so there is no collision). Stored are the display and interaction options, the disk options, both shortcuts and the power-off parameters. On load every value is checked against its valid range; implausible values fall back to the default.

The Wi-Fi credentials live in a namespace of **their own**, `c64unet` (see the *Wi-Fi subsystem* chapter), so they survive a *Factory Reset*. `build_env.h` only supplies the initial values as long as nothing has been stored there.

Project structure

```
M5Dial-C64uRemote/
├── platformio.ini            board, libraries, partition, upload
├── README.md
├── LICENSE                   MIT - Karl Prosser, Martin Oswald
├── wifi.txt.example          template for /wifi.txt on the SD card
├── .vscode/                  recommended extensions, editor settings
├── docs-src/                 markdown sources of the manuals
├── doc/                      finished manuals (PDF)
├── src/                      German edition
│   ├── main.cpp              entire firmware
│   ├── build_env.h           credentials (do not version)
│   ├── build_env.h.example   template
│   └── 1MHz_logo_rgb565.h    logo, 240 × 135, RGB565, PROGMEM
└── src-en/
    └── main.cpp              English edition, identical code
```

Troubleshooting

Symptom	Cause / remedy
SET build_env.h at startup	credentials missing
KEIN NFC-LESER	check the serial output: `RFID (I2C 0x28) VersionReg = ….` `0x00`/`0xFF` means no contact with the internal bus
KEINE SD-KARTE	wiring, module supply voltage, FAT32, cable length

Symptom	Cause / remedy
KEINE SD-KARTE after wiring per docs up to v1.2.1	swap the yellow and white wire on Port B: yellow = MISO (G2), white = CS (G1)
Status dot yellow (*AUTH?*)	`C64U_TARGET_PASSWORD` is wrong
Invalid type when mounting	`type`/`mode` belong in the query, not as a multipart text field
LADEN ZU LANG	loading exceeds 180 s; check the image or set *Disk Action* to *Mnt+Reset*
Tag readable, file missing	the path on the tag does not match the SD card; *NFC-Info* shows it
Display stutters	set *Effect* to `Static` or shorten *Effect Time*
`Kein Speicher fuer den Offscreen-Puffer` in the log	heap is tight; the firmware keeps running but draws directly to the display

References

- C64 Ultimate ReST API: `https://1541u-documentation.readthedocs.io`
- Original project: `https://github.com/ReadyOS-C64/C64uRemote`
- M5Dial: `https://docs.m5stack.com/en/core/M5Dial`
- M5Dial library: `https://github.com/m5stack/M5Dial`
- Tag format: TeensyROM NFC loader, Zaparoo/TapTo

License

C64uRemote is released under the **MIT License**. The full text is in the file `LICENSE` in the project root.

It originates from **C64uRemote by Karl Prosser (@klumsy)**, https://github.com/ReadyOS-C64/C64uRemote, which he published under the MIT License. This version is a derivative work and is released under the same terms:

- Copyright (c) 2026 Karl Prosser – original project
- Copyright (c) 2026 Martin Oswald (@mad, https://1MHz.de) – port and extensions

The MIT License allows you to use, modify and redistribute the software, including commercially. The only condition: **the copyright notice and the license text must be kept** and included with every copy. There is no warranty and no liability.

The libraries used carry their own licenses: M5Unified and M5GFX (MIT, © M5Stack), ArduinoJson (MIT, © Benoit Blanchon) and MFRC522_I2C (https://github.com/kkloesener/MFRC522_I2C). PlatformIO fetches them at build time.

How this was made

The port, the extensions and these manuals were written with the help of Claude (Anthropic). Concept, idea, hardware decisions and every test on real devices: Martin Oswald (@mad).